WEB TASARIMI VE PROGRAMLAMA

LAB-6

React Temelleri · Component, Props/State, Forms · Ara Checkpoint #1

İçindekiler

1 LAB-6 Öğrenme Hedefleri **2**

2 React Component Mimarisi **2**

3 Props: Component'ler Arası Veri Aktarımı **4**

4 useState ile State Yönetimi **7**

5 useMemo ile Performans Optimizasyonu **10**

6 Controlled Form (Kontrollü Form) **12**

7 Liste Render ve Key Kavramı **19**

8 Ara Checkpoint #1: Bütünleşik Uygulama **22**

9 Git İş Akışı ve PR Tabanlı Kod İnceleme **33**

10 Sınıf İçi Demo ve Değerlendirme **36**

11 Teslim Kontrol Listesi ve Değerlendirme Rubriği **37**

11.1 Değerlendirme Rubriği 37

12 Sık Sorulan Sorular **38**

13 Kendini Test Et **38**

14 Ek Kaynaklar ve İleri Okuma **39**

15 Gelecek Hafta: Nereye Gidiyoruz? **39**

1 LAB-6 Öğrenme Hedefleri

Önceki laboratuvarlarda geliştirme ortamını kurmuş (LAB-1), semantik HTML yapısını oluşturmuş (LAB-2), responsive CSS ile düzenlemiş (LAB-3), Tailwind CSS ile component tabanlı bir UI kütüphanesi inşa etmiş (LAB-4) ve JavaScript/TypeScript temelleriyle birlikte state mantığını kavramıştın (LAB-5). Artık veri modelin, filtreleme fonksiyonların ve tip tanımların hazır; şimdi sıra tüm bu yapı taşlarını **React component'leri** ile bir araya getirmeye geldi.

Bu laboratuvarda React'in temel yapı taşlarını öğreneceksin: **component** (bileşen) mimarisi, **props** ile veri aktarımı, **state** ile durum yönetimi, **controlled form** yapıları ve **liste render** kavramı. Ayrıca bu hafta ilk **Ara Checkpoint**'i tamamlayarak 1–6. haftalarda öğrendiğin tüm becerileri bir arada sergileyeceksin.

Bu laboratuvarın sonunda aşağıdaki becerilere sahip olacaksın:

1. **React component oluşturmak:** Fonksiyonel component yazarak UI'ı bağımsız, yeniden kullanılabilir parçalara bölmek.
2. **Props ile veri aktarmak:** Parent'tan child'a veri gönderme, TypeScript ile prop tiplerini tanımlama.
3. **useState ile state yönetmek:** Component düzeyinde durum oluşturma ve güncelleme.
4. **useMemo ile performans optimizasyonu:** Pahalı hesaplamaları önbelleğe almak.
5. **Controlled form oluşturmak:** Form elemanlarını state ile kontrol etmek, form verilerini toplamak ve doğrulamak.
6. **Liste render ve key kavramını uygulamak:** Dizi verisini `.map()` ile JSX'e dönüştürmek, `key` prop'unun neden kritik olduğunu anlamak.
7. **Checkpoint #1 demosunu tamamlamak:** Portföy/landing sayfası + proje listesi + filtreleme + responsive tasarım bütünleşik çalışır durumda.

Benzetme ile Anlama

LAB-1–5'te binanın temelini, duvarlarını, boyasını, elektrik tesisatını ve su borularını ayrı ayrı döşedik. Şimdi **iç mimari** aşamasındayız: Odaları (component), kapıları (props), ışık anahtarlarını (state) ve priz panellerini (form) yerleştiriyoruz. Her oda bağımsız ama birlikte bir **yaşanabilir ev** oluşturuyor. Checkpoint #1, binanın ilk "oturma izni" kontrolüdür — her şey çalışıyor mu, güvenli mi, yaşanabilir mi?

Sırada Ne Var?

Hedeflerimiz belirlendi. İlk adım olarak React'in temel yapı taşı olan **component** kavramını derinlemesine inceleyelim.

2 React Component Mimarisi

React'te her şey **component**'tir. Bir buton, bir form, bir kart, hatta tüm sayfa bile birer component'tir. Component'ler, UI'ın bağımsız, yeniden kullanılabilir ve test edilebilir

parçalarıdır.

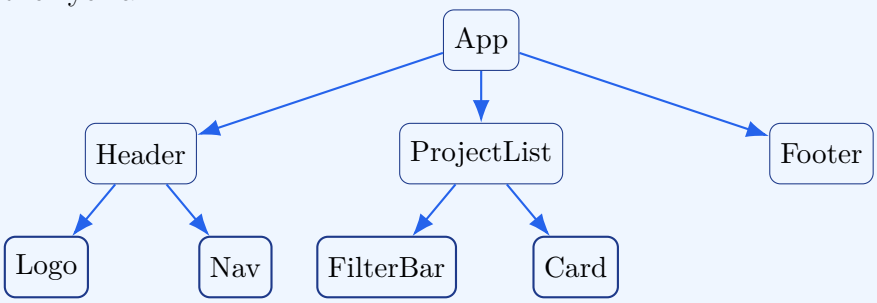
Teorik Arka Plan

Component nedir?
Bir React component, **JSX döndüren bir JavaScript fonksiyonudur**. İsmi büyük harfle başlar (PascalCase). Her component kendi "dünyasını" yönetir — kendi state'i, kendi görünümü, kendi mantığı vardır.

```
// En basit component: parametre almaz, JSX doner
function Welcome() {
  return <h1>Merhaba, React!</h1>;
}

// Kullanim: HTML tag'i gibi yazilir
function App() {
  return (
    <div>
      <Welcome />
      <Welcome />
      <Welcome />
    </div>
  );
}
// Ekranda 3 adet "Merhaba, React!" gorulur
```

Component ağacı (Component Tree):
React uygulamaları bir **ağaç** yapısındadır. En üstte `App` component'i bulunur; onun altında sayfalar, onların altında bölümler, en altta ise buton ve input gibi atomik component'ler yer alır.



Her kutu bir component'tir. Oklar parent → child ilişkisini gösterir. Veri (props) yukarıdan aşağıya akar.

Derinlemesine Anlama

Neden component bazlı mimari?

- **Yeniden kullanılabilirlik:** Bir `Card` component'i yazarsın, 50 yerde kullanırsın. Değişiklik yaparsan tek yerde yaparsın.
- **Ayrıştırma (Separation of Concerns):** Her component yalnızca kendi işini yapar. `FilterBar` filtreleme ile ilgilenir, `Card` görüntüleme ile.
- **Test edilebilirlik:** Her component bağımsız test edilebilir — 1000 satırlık bir sayfayı test etmek yerine 50 satırlık bir component'i test edersin.
- **Paralel çalışma:** Bir geliştirici `Header`'ı, başka biri `ProjectList`'i yazabilir —

birbirlerini beklemezler.
- **Okunabilirlik:** `<ProjectList />` yazdığında ne olduğu anlaşılır. `document.getElementById("list").innerHTML = ...` yazdığında değil.

Gerçek Hayatta Karşılığı

Component mimarisini bir **LEGO seti** gibi düşün. Her LEGO parçası (component) kendi başına bir anlam ifade eder — bir tekerlek, bir pencere, bir çatı. Bu parçaları birleştirerek bir ev, bir araba veya bir uzay gemisi yapabilirsin. Parçalar değişmez ama **kombinasyonlar** sınırsızdır. Aynı `Button` component'ini giriş formunda da, ayarlar sayfasında da, modal'da da kullanabilirsin.

Dikkat – Sık Yapılan Hatalar

- Component isimleri **mutlaka** büyük harfle başlamalıdır: `ProjectCard`, `FilterBar`. Küçük harfle başlarsa React bunu HTML elemanı sanır.
- Her component **tek bir kök eleman** döndürmelidir. Birden fazla eleman döndürmek istiyorsan `<div>` veya `<>...</>` (Fragment) ile sar.
- Component'ler **saf fonksiyonlar** (pure functions) gibi davranmalıdır: Aynı props verildiğinde aynı JSX çıkmalıdır. Render sırasında dış değişkenleri değiştirme (side effect yapma).

Sırada Ne Var?

Component kavramını öğrendik. Şimdi component'ler arasında **veri aktarımını** sağlayan **props** mekanizmasını inceleyelim.

3 Props: Component'ler Arası Veri Aktarımı

Props (properties), bir parent component'ten child component'e veri aktarmanın yoludur. HTML elemanlarının attribute'ları gibi düşünebilirsin — ama çok daha güçlü, çünkü her türlü JavaScript değerini (string, number, array, object, function, hatta başka component'ler) aktarabilirsin.

Teorik Arka Plan

Props kullanımı:

```
// --- 1. Props interface'i tanimla ---
interface ProjectCardProps {
  title: string;
  description: string;
  tech: string[];
  year: number;
  featured?: boolean; // opsiyonel
}

// --- 2. Component, props'u parametre olarak alir ---
function ProjectCard({
```

```
  title,
  description,
  tech,
  year,
  featured = false, // varsayilan deger
}: ProjectCardProps) {
  return (
    <div className={`rounded-lg border p-4
      ${featured
        ? "border-blue-500 bg-blue-50"
        : "border-gray-200"}`}>
      <h3 className="text-lg font-bold">{title}</h3>
      <p className="text-gray-600 mt-1">
        {description}
      </p>
      <div className="flex gap-2 mt-3">
        {tech.map(t => (
          <span key={t}
            className="bg-gray-100 text-xs
              px-2 py-0.5 rounded-full">
            {t}
          </span>
        ))}
      </div>
      <p className="text-sm text-gray-400 mt-2">
        {year}
      </p>
    </div>
  );
}
```

```
// --- 3. Parent'tan kullanim ---
function App() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-2
      gap-4 p-8">
      <ProjectCard
        title="E-Ticaret Platformu"
        description="React ile tam kapsamli uygulama"
        tech={["React", "Node.js", "MongoDB"]}
        year={2024}
        featured={true}
      />
      <ProjectCard
        title="Blog Uygulamasi"
        description="Next.js ile SEO dostu blog"
        tech={["Next.js", "TypeScript"]}
        year={2024}
      />
    </div>
```

```
  );
}
```

children prop (özel prop):

```
// children: Component'in acilis ve kapanis
// tag'leri arasindaki icerik
interface CardProps {
  title: string;
  children: ReactNode; // herhangi bir JSX olabilir
}

function Card({ title, children }: CardProps) {
  return (
    <div className="rounded-lg border p-4">
      <h3 className="font-bold">{title}</h3>
      <div className="mt-2">{children}</div>
    </div>
  );
}

// Kullanim
<Card title="Proje Detayi">
  <p>Bu bir aciklama metnidir.</p>
  <button>Detay Gor</button>
</Card>
```

Derinlemesine Anlama

Props'un temel kuralları:

1. **Props tek yönlüdür (one-way data flow):** Veri **yalnızca** parent'tan child'a akar. Child, aldığı props'u **değiştiremez** — props `readonly`'dir.
2. **Props immutable'dır:** Bir component aldığı prop'u mutate edemez. `props.title = "Yeni"` yazmak yasaktır ve hata verir.
3. **Callback props:** Child'dan parent'a "bilgi göndermek" için parent bir **fonksiyon** gönderir, child bu fonksiyonu çağırır:

```
// Parent
function App() {
  const handleDelete = (id: number) => {
    console.log("Silinecek proje:", id);
  };

  return (
    <ProjectCard
      id={1}
      title="E-Ticaret"
      onDelete={handleDelete}
    />
  );
```

```
}

// Child
interface ProjectCardProps {
  id: number;
  title: string;
  onDelete: (id: number) => void;
}

function ProjectCard({
  id, title, onDelete
}: ProjectCardProps) {
  return (
    <div>
      <h3>{title}</h3>
      <button onClick={() => onDelete(id)}>
        Sil
      </button>
    </div>
  );
}
```

4. **Tip güvenliği:** TypeScript interface'i ile props'un şeklini tanımlarsan, yanlış tip veya eksik prop gönderdiğinde **derleme zamanında** hata alırsın.

Dikkat – Sık Yapılan Hatalar

- Props'u doğrudan değiştirmeye çalışma: `title = "Yeni"` **yasaktır**. Değişiklik gerekiyorsa parent'ın state'ini güncelle.
- Boolean prop'larda `true` değerini kısaltabilirsin: `<Card featured />` ile `<Card featured={true} />` aynıdır.
- `children` prop'u her zaman açıkça tip tanımla: `ReactNode` (her türlü JSX), `string` (yalnızca metin) veya `React.ReactElement` (yalnızca element).

Sırada Ne Var?

Props ile component'ler arası veri aktarımını öğrendik. Şimdi React'in en güçlü özelliğine geçiyoruz: **useState** ile state yönetimi.

4 useState ile State Yönetimi

Props dışarıdan gelen (parent'ın kontrol ettiği) veridir. **State** ise component'in **kendi iç durumudur** — kullanıcı etkileşimiyle değişir ve her değişiklikte component yeniden render edilir.

Teorik Arka Plan

useState Hook'u:

```
import { useState } from "react";

function Counter() {
  // [mevcut deger, guncelleme fonksiyonu]
  const [count, setCount] = useState(0);
  // useState(0): baslangic degeri 0

  return (
    <div className="flex items-center gap-4">
      <button
        onClick={() => setCount(count - 1)}
        className="px-3 py-1 bg-red-500
          text-white rounded">
        -
      </button>

      <span className="text-2xl font-bold">
        {count}
      </span>

      <button
        onClick={() => setCount(count + 1)}
        className="px-3 py-1 bg-green-500
          text-white rounded">
        +
      </button>
    </div>
  );
}
```

Ne oluyor, adım adım:

1. `useState(0)` çağrısı, `count` değişkenini `0` başlangıç değeriyle oluşturur.
2. Kullanıcı "+" butonuna tıklar → `setCount(count + 1)` çağrılır.
3. React, state'in değiştiğini algılar ve component'i **yeniden render** eder.
4. Yeni render'da `count` artık `1`'dir — ekrandaki sayı güncellenir.

TypeScript ile tipli state:

```
// Basit tipler: TypeScript otomatik cikarir
const [count, setCount] = useState(0);
// count: number

const [name, setName] = useState("");
// name: string

const [isOpen, setIsOpen] = useState(false);
// isOpen: boolean

// Karmasik tipler: Acikca belirt
const [projects, setProjects] =
  useState<Project[]>([]);
```

```
const [error, setError] =
  useState<string | null>(null);

const [category, setCategory] =
  useState<Category | "all">("all");
```

State güncelleme kuralları:

```
// --- YANLIS: Dogrudan mutasyon ---
const [items, setItems] = useState(["a", "b"]);
items.push("c"); // YANLIS! State mutate edildi
setItems(items);  // React degisikligi FARKETMEZ

// --- DOGRU: Yeni referans olustur ---
setItems([...items, "c"]); // spread ile yeni dizi

// --- Obje state guncelleme ---
const [filters, setFilters] = useState({
  search: "",
  category: "all"
});

// YANLIS:
filters.search = "react";
setFilters(filters);

// DOGRU: Spread ile yeni obje
setFilters({ ...filters, search: "react" });

// --- Onceki deger uzerinden guncelleme ---
// YANLIS (stale closure riski):
setCount(count + 1);
setCount(count + 1); // hala +1 (2 degil!)

// DOGRU (updater function):
setCount(prev => prev + 1);
setCount(prev => prev + 1); // dogru: +2
```

Benzetme ile Anlama

State'i bir **beyaz tahta** gibi düşün. Component'in üzerindeki beyaz tahtada mevcut durum yazılıdır. Kullanıcı bir butona tıkladığında tahtayı **silip yeniden yazarsın** (yeni state). React bu yeni tahtaya bakarak ekranı günceller. Önemli olan: Tahtanın üzerini silmeden üstüne yazamazsın — her zaman **temiz bir kopya** oluşturmalısın (immutability).

Dikkat – Sık Yapılan Hatalar

- State'i **doğrudan mutate etme**. Dizi için `push/pop/splice` yerine `[...arr, newItem]` veya `arr.filter(...)` kullan. Obje için `obj.key = val` yerine `{...obj, key: val}` kullan.
- `setState` **asenkrondur** — çağırdıktan hemen sonra yeni değeri okuyamazsın. Yeni değeri bir sonraki render'da görürsün.
- Önceki state'e bağlı güncellemelerde **updater function** kullan: `setCount(prev => prev + 1)`. Aksi halde "stale closure" (bayat değer) sorunu yaşarsın.
- Çok fazla `useState` varsa (>4–5), state'i bir obje olarak gruplamayı veya `useReducer` hook'unu kullanmayı düşün.

Sırada Ne Var?

State yönetimini öğrendik. Şimdi pahalı hesaplamaları (filtreleme, sıralama) optimize etmek için **useMemo** hook'unu inceleyelim.

5 useMemo ile Performans Optimizasyonu

Her state değişikliğinde component yeniden render edilir ve tüm JavaScript kodu tekrar çalışır. Filtreleme ve sıralama gibi işlemler binlerce öğede her render'da tekrar hesaplanırsa performans düşer. `useMemo`, sonucu **önbelleğe alır** ve yalnızca bağımlılıkları değiştiğinde yeniden hesaplar.

Teorik Arka Plan

useMemo Hook'u:

```
import { useState, useMemo } from "react";

function ProjectList() {
  const [projects, setProjects] =
    useState<Project[]>([]);
  const [search, setSearch] = useState("");
  const [category, setCategory] =
    useState<Category | "all">("all");
  const [sortField, setSortField] =
    useState<SortField>("year");
  const [sortOrder, setSortOrder] =
    useState<SortOrder>("desc");

  // useMemo: Yalnizca bagimliliklar degistiginde
  // yeniden hesaplar
  const filteredProjects = useMemo(() => {
    console.log("Filtreleme hesaplaniyor...");
    return applyFilters(
      projects, search, category,
      sortField, sortOrder
    );
```

```
  }, [projects, search, category,
      sortField, sortOrder]);
  // ^ Bagimlilik dizisi: Bu degerlerden biri
  //   degistiginde yeniden hesapla

  // Her render'da degil, YALNIZCA gerektiginde
  // "Filtreleme hesaplaniyor..." gorulur

  return (
    <div>
      <p>{filteredProjects.length} proje bulundu</p>
      {filteredProjects.map(p => (
        <ProjectCard key={p.id} {...p} />
      ))}
    </div>
  );
}
```

useMemo ne zaman kullanılır?

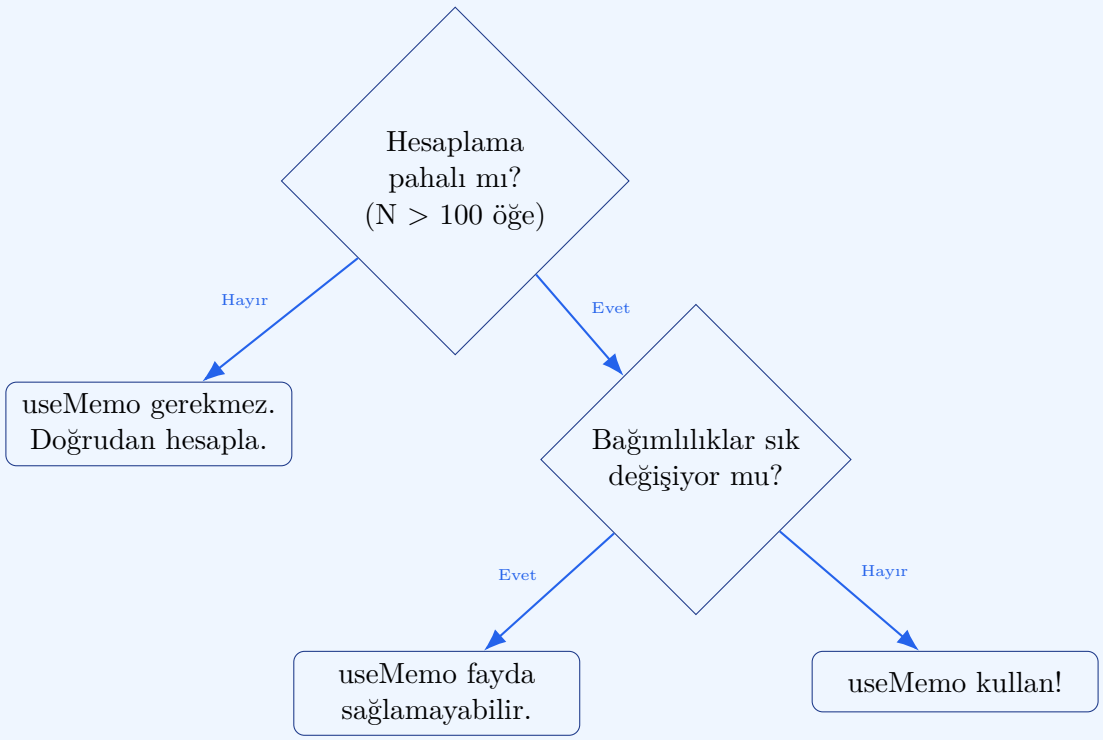
- Büyük dizilerde filtreleme, sıralama, gruplama gibi **pahalı hesaplamalar**.
- Birden fazla state'ten **türetilmiş veri** (derived data) hesaplama.
- Referans eşitliği önemli olduğunda (child component'e obje/dizi prop olarak geçerken).

useMemo ne zaman KULLANILMAZ?

- Basit hesaplamalar (`count + 1`, string birleştirme).
- State'i doğrudan döndüren ifadeler.
- "Her yere koy, performans artar" düşüncesiyle — `useMemo`'nun kendisi de bir maliyettir (bağımlılık karşılaştırması).

Derinlemesine Anlama

useMemo vs hesaplama: Karar ağacı



Pratik kural: Önce `useMemo` **olmadan** yaz. Performans sorunu fark edersen ekle. Erken optimizasyon, kötülüğün anasıdır.

Dikkat – Sık Yapılan Hatalar

- `useMemo`'nun bağımlılık dizisini **doğru** yaz. Eksik bağımlılık: eski (stale) veri. Fazla bağımlılık: gereksiz yeniden hesaplama.
- `useMemo` bir **öneri** (hint), garanti değildir. React bazı durumlarda önbelleği temizleyebilir. Side effect için kullanma; side effect için `useEffect` kullan.
- `useMemo` ile `useCallback` karıştırma. `useMemo` bir **değeri**, `useCallback` bir **fonksiyonu** önbelleğe alır.

Sırada Ne Var?

Performans optimizasyonunu kavradık. Şimdi kullanıcıdan veri toplamak için **controlled form** yapısını öğrenelim.

6 Controlled Form (Kontrollü Form)

HTML'de form elemanları (`<input>`, `<select>`, `<textarea>`) kendi iç durumlarını (değer, seçim) kendileri yönetir. React'te ise form elemanlarının durumunu **state ile kontrol ederiz** — buna **controlled component** denir.

Teorik Arka Plan

Controlled vs Uncontrolled:

```
// --- UNCONTROLLED (React bilmiyor) ---
function UncontrolledForm() {
```

```
  const handleSubmit = (e: FormEvent) => {
    e.preventDefault();
    // DOM'dan degeri oku (React bilmiyor!)
    const input = document.getElementById("name");
    console.log(input.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input id="name" />
      <button type="submit">Gonder</button>
    </form>
  );
}

// --- CONTROLLED (React yonetiyor) ---
function ControlledForm() {
  const [name, setName] = useState("");

  const handleSubmit = (e: FormEvent) => {
    e.preventDefault();
    // State'ten oku (React her an biliyor!)
    console.log(name);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={e => setName(e.target.value)}
      />
      <button type="submit">Gonder</button>
    </form>
  );
}
```

Controlled component'te form elemanının `value` özelliği state'ten gelir ve `onChange` ile state güncellenir. Bu sayede React her an formun durumunu bilir.

Uygulama Adımı

Uygulama-1: İletişim Formu Component'i

`src/components/ContactForm.tsx` dosyası oluştur:

```
import { useState, type FormEvent } from "react";

// --- Form veri modeli ---
interface ContactFormData {
  name: string;
  email: string;
```

```
  subject: string;
  message: string;
}

// --- Form hata modeli ---
interface FormErrors {
  name?: string;
  email?: string;
  subject?: string;
  message?: string;
}

// --- Baslangic degerleri ---
const initialFormData: ContactFormData = {
  name: "",
  email: "",
  subject: "",
  message: "",
};

export default function ContactForm() {
  // Form state
  const [formData, setFormData] =
    useState<ContactFormData>(initialFormData);
  const [errors, setErrors] =
    useState<FormErrors>({});
  const [isSubmitting, setIsSubmitting] =
    useState(false);
  const [submitSuccess, setSubmitSuccess] =
    useState(false);

  // --- Dogrulama (Validation) ---
  function validate(
    data: ContactFormData
  ): FormErrors {
    const newErrors: FormErrors = {};

    if (!data.name.trim()) {
      newErrors.name = "Ad soyad zorunludur.";
    } else if (data.name.trim().length < 2) {
      newErrors.name =
        "Ad soyad en az 2 karakter olmalidir.";
    }

    if (!data.email.trim()) {
      newErrors.email = "E-posta zorunludur.";
    } else if (
      !/^[^\s@]+@[^\s@]+\.[^\s@]+$/
        .test(data.email)
    ) {
```

```
      newErrors.email =
        "Gecerli bir e-posta adresi giriniz.";
    }

    if (!data.subject.trim()) {
      newErrors.subject = "Konu zorunludur.";
    }

    if (!data.message.trim()) {
      newErrors.message = "Mesaj zorunludur.";
    } else if (data.message.trim().length < 10) {
      newErrors.message =
        "Mesaj en az 10 karakter olmalidir.";
    }

    return newErrors;
  }

  // --- Tek alan guncelleme ---
  function handleChange(
    field: keyof ContactFormData,
    value: string
  ) {
    setFormData(prev => ({
      ...prev,
      [field]: value,
    }));
    // Hata mesajini temizle
    if (errors[field]) {
      setErrors(prev => ({
        ...prev,
        [field]: undefined,
      }));
    }
  }

  // --- Form gonderme ---
  async function handleSubmit(e: FormEvent) {
    e.preventDefault();

    // Dogrula
    const newErrors = validate(formData);
    if (Object.keys(newErrors).length > 0) {
      setErrors(newErrors);
      return;
    }

    setIsSubmitting(true);
    try {
      // Simule edilmis API cagirisi
```

```
      await new Promise(
        resolve => setTimeout(resolve, 1000)
      );
      console.log("Form verisi:", formData);
      setSubmitSuccess(true);
      setFormData(initialFormData);
    } catch {
      alert("Gonderim basarisiz. Tekrar deneyin.");
    } finally {
      setIsSubmitting(false);
    }
  }

  if (submitSuccess) {
    return (
      <div className="bg-green-50 border
        border-green-200 rounded-lg p-6
        text-center">
        <p className="text-green-800 font-medium">
          Mesajiniz basariyla gonderildi!
        </p>
        <button
          onClick={() => setSubmitSuccess(false)}
          className="mt-4 text-sm text-green-600
            underline">
          Yeni mesaj gonder
        </button>
      </div>
    );
  }

  return (
    <form onSubmit={handleSubmit}
      className="space-y-4 max-w-lg" noValidate>

      {/* Ad Soyad */}
      <div>
        <label htmlFor="name"
          className="block text-sm font-medium
            mb-1">
          Ad Soyad
        </label>
        <input id="name" type="text"
          value={formData.name}
          onChange={e =>
            handleChange("name", e.target.value)}
          className={`w-full border rounded-lg
            px-3 py-2 ${errors.name
              ? "border-red-500" : "border-gray-300"
            }`}
```

```
          placeholder="Adiniz Soyadiniz" />
        {errors.name && (
          <p className="text-red-500 text-sm mt-1">
            {errors.name}
          </p>
        )}
      </div>

      {/* E-posta */}
      <div>
        <label htmlFor="email"
          className="block text-sm font-medium
            mb-1">
          E-posta
        </label>
        <input id="email" type="email"
          value={formData.email}
          onChange={e =>
            handleChange("email", e.target.value)}
          className={`w-full border rounded-lg
            px-3 py-2 ${errors.email
              ? "border-red-500" : "border-gray-300"
            }`}
          placeholder="ornek@mail.com" />
        {errors.email && (
          <p className="text-red-500 text-sm mt-1">
            {errors.email}
          </p>
        )}
      </div>

      {/* Konu */}
      <div>
        <label htmlFor="subject"
          className="block text-sm font-medium
            mb-1">
          Konu
        </label>
        <select id="subject"
          value={formData.subject}
          onChange={e =>
            handleChange("subject", e.target.value)}
          className={`w-full border rounded-lg
            px-3 py-2 ${errors.subject
              ? "border-red-500" : "border-gray-300"
            }`}>
          <option value="">Konu seciniz...</option>
          <option value="genel">Genel</option>
          <option value="destek">
            Teknik Destek
```

```
          </option>
          <option value="oneri">Oneri</option>
          <option value="isbirligi">
            Is Birligi
          </option>
        </select>
        {errors.subject && (
          <p className="text-red-500 text-sm mt-1">
            {errors.subject}
          </p>
        )}
      </div>

      {/* Mesaj */}
      <div>
        <label htmlFor="message"
          className="block text-sm font-medium
            mb-1">
          Mesaj
        </label>
        <textarea id="message" rows={5}
          value={formData.message}
          onChange={e =>
            handleChange("message", e.target.value)}
          className={`w-full border rounded-lg
            px-3 py-2 resize-y ${errors.message
              ? "border-red-500" : "border-gray-300"
            }`}
          placeholder="Mesajinizi yaziniz..." />
        {errors.message && (
          <p className="text-red-500 text-sm mt-1">
            {errors.message}
          </p>
        )}
      </div>

      {/* Gonder butonu */}
      <button type="submit"
        disabled={isSubmitting}
        className="w-full bg-blue-600 text-white
          py-2 rounded-lg font-medium
          hover:bg-blue-700 transition-colors
          disabled:opacity-50
          disabled:cursor-not-allowed">
        {isSubmitting
          ? "Gonderiliyor..." : "Gonder"}
      </button>
    </form>
  );
}
```

Derinlemesine Anlama

Controlled form'un avantajları:

- **Anlık doğrulama:** Her karakter girildiğinde hata kontrolü yapabilirsin (e-posta formatı, minimum uzunluk).
- **Koşullu görünüm:** Seçime göre farklı alanlar gösterebilirsin (örn. "İş birliği" seçildiğinde şirket adı alanı açılır).
- **Gönderim engelleme:** Form geçersizse buton `disabled` olur.
- **Programatik kontrol:** Formu kodla doldurabilir, temizleyebilir veya varsayılan değerlerle açabilirsin.
- **Test edilebilirlik:** State'i test kodundan ayarlayıp form davranışını test edebilirsin — DOM'a dokunmana gerek yok.

Dikkat – Sık Yapılan Hatalar

- `value` prop'u verip `onChange` vermezsen form alanı **yazılamaz** (frozen) olur. React konsolda uyarı verir.
- `e.preventDefault()` çağrısını **unutma**. Yoksa form gönderildiğinde sayfa yenilenir ve tüm state kaybolur.
- Her alan için ayrı `useState` yerine tek bir obje state'i kullan (`formData` gibi). Böylece form büyüdükçe state yönetimi karmaşıklaşmaz.
- Doğrulama mantığını component **dışında** tanımla — test edilmesi ve yeniden kullanılması kolaylaşır.

Sırada Ne Var?

Controlled form yapısını öğrendik. Şimdi dizi verisini ekranda listelemek için **liste render** ve **key** kavramını derinlemesine inceleyelim.

Teorik Arka Plan

Temel liste render:

```
const technologies = [
  "React", "TypeScript", "Tailwind", "Vite"
];

function TechList() {
  return (
    <ul className="space-y-2">
      {technologies.map(tech => (
        <li key={tech}
          className="bg-gray-100 px-3 py-1
            rounded">
```

```
          {tech}
        </li>
      ))}
    </ul>
  );
}
```

Obje dizisi ile liste render:

```
interface Project {
  id: number;
  title: string;
  description: string;
  tech: string[];
}

function ProjectGrid({
  projects
}: {
  projects: Project[]
}) {
  if (projects.length === 0) {
    return (
      <p className="text-gray-500 text-center">
        Gosterilecek proje bulunamadi.
      </p>
    );
  }

  return (
    <div className="grid grid-cols-1
      md:grid-cols-2 lg:grid-cols-3 gap-6">
      {projects.map(project => (
        <ProjectCard key={project.id}
          title={project.title}
          description={project.description}
          tech={project.tech}
        />
      ))}
    </div>
  );
}
```

key prop'u nedir?

key, React'e listedeki her elemanın **kimliğini** bildirir. React, key'leri kullanarak hangi elemanların eklendiğini, silindiğini veya yer değiştirdiğini anlar ve yalnızca **değişen** elemanları DOM'da günceller.

```
// DOGRU: Benzersiz ve sabit bir deger
{projects.map(p => (
  <Card key={p.id} title={p.title} />
))}
```

```
// YANLIS: Dizi indeksi
{projects.map((p, index) => (
  <Card key={index} title={p.title} />
))}
// Siralama degistiginde React karisir!

// YANLIS: Rastgele deger
{projects.map(p => (
  <Card key={Math.random()} title={p.title} />
))}
// Her render'da yeni key = her seferinde
// tum DOM yeniden olusturulur!
```

Derinlemesine Anlama

key prop'unun ardındaki mekanizma: Reconciliation
React, state değiştiğinde yeni bir **Virtual DOM** ağacı oluşturur ve eskisiyle karşılaştırır (diffing). Liste elemanlarında bu karşılaştırmayı `key` ile yapar:

1. **Aynı key, aynı pozisyon:** Eleman **güncellenir** (DOM'dan kaldırılmaz, yalnızca değişen prop'lar uygulanır).
2. **Yeni key:** Eleman DOM'a **eklenir** (mount).
3. **Kaybolan key:** Eleman DOM'dan **kaldırılır** (unmount).
4. **Aynı key, farklı pozisyon:** Eleman DOM'da **taşınır** (reorder) — yeniden oluşturulmaz.

index kullanmanın sorunu:
Diyelim ki listede 3 öğe var: A (key=0), B (key=1), C (key=2). B'yi silersek: A (key=0), C (key=1). React, key=1'in "B iken C oldu" sanır ve gereksiz güncelleme yapar. Eğer key={item.id} kullansaydık: A (key=1), C (key=3) — React doğrudan B'yi kaldırır, A ve C'ye dokunmaz.

Dikkat – Sık Yapılan Hatalar

- **key** prop'u component'e prop olarak **geçmez**. `key={p.id}` yazdığında component içinden `props.key` ile erişemezsin. Eğer id'ye ihtiyacın varsa ayrıca `id={p.id}` gönder.
- `key` yalnızca **kardeşler arasında** (siblings) benzersiz olmalıdır — farklı listeler arasında aynı key olabilir.
- Dizi indeksini key olarak yalnızca **statik, sıralanmayan, elemanları eklenip silinmeyen** listelerde kullanabilirsin. Diğer tüm durumlarda benzersiz bir `id` kullan.
- Boş liste durumunu **her zaman** kontrol et ve kullanıcıya uygun bir mesaj göster: "Gösterilecek proje bulunamadı."

Sırada Ne Var?

Liste render ve key kavramını derinlemesine öğrendik. Şimdi tüm LAB-6 bilgilerini birleştirerek Checkpoint #1 uygulamasını inşa edeceğiz.

Ara Checkpoint #1

Checkpoint #1 – Demo Gereksinimleri:

1. **Portföy / Landing Sayfası:** Hero bölümü, hakkımda, yetenekler, iletişim formu.
2. **Proje Listesi:** JSON'dan veri çekme, kart grid yapısı.
3. **Filtreleme ve Sıralama:** Arama, kategori filtresi, yıla/başlığa göre sıralama.
4. **Responsive Tasarım:** Mobil, tablet ve masaüstünde düzgün görünüm.
5. **İletişim Formu:** Controlled form, doğrulama (validation), gönderim durumu.
6. **Erişilebilirlik (a11y):** Semantik HTML, label'lar, keyboard navigasyon.
7. **TypeScript:** `any` kullanılmamış, tüm tipler tanımlı.

Uygulama Adımı

Uygulama-2: Component Yapısını Oluştur

Projedeki dosya yapısı şu şekilde olmalıdır:

```
src/
  components/
    layout/
      Header.tsx
      Footer.tsx
    ui/
      Button.tsx
      Input.tsx
      Card.tsx
      Alert.tsx
    sections/
      Hero.tsx
      About.tsx
      Skills.tsx
      ProjectList.tsx
      ContactSection.tsx
    forms/
      ContactForm.tsx
      ProjectFilter.tsx
  types/
    project.ts
  services/
    projectService.ts
  utils/
    projectHelpers.ts
  App.tsx
  main.tsx
public/
  data/
```

```
    projects.json
```

Uygulama Adımı

Uygulama-3: Header Component'i

src/components/layout/Header.tsx:

```
import { useState } from "react";

const navLinks = [
  { href: "#hero", label: "Ana Sayfa" },
  { href: "#about", label: "Hakkimda" },
  { href: "#projects", label: "Projeler" },
  { href: "#contact", label: "Iletisim" },
];

export default function Header() {
  const [menuOpen, setMenuOpen] = useState(false);

  return (
    <header className="sticky top-0 z-50
      bg-white/80 dark:bg-gray-900/80
      backdrop-blur-sm border-b
      border-gray-200 dark:border-gray-800">
      <nav className="max-w-6xl mx-auto px-4
        h-16 flex items-center
        justify-between">

        {/* Logo */}
        <a href="#hero"
          className="text-xl font-bold
            text-blue-600">
          Portfolyo
        </a>

        {/* Desktop nav */}
        <ul className="hidden md:flex gap-6">
          {navLinks.map(link => (
            <li key={link.href}>
              <a href={link.href}
                className="text-gray-600
                  dark:text-gray-300
                  hover:text-blue-600
                  transition-colors">
                {link.label}
              </a>
            </li>
          ))}
        </ul>

```

```
        {/* Mobil menu butonu */}
        <button
          onClick={() => setMenuOpen(!menuOpen)}
          className="md:hidden p-2"
          aria-label="Menu"
          aria-expanded={menuOpen}>
          <span className="block w-6 h-0.5
            bg-gray-600 mb-1" />
          <span className="block w-6 h-0.5
            bg-gray-600 mb-1" />
          <span className="block w-6 h-0.5
            bg-gray-600" />
        </button>
      </nav>

      {/* Mobil menu */}
      {menuOpen && (
        <ul className="md:hidden border-t
          border-gray-200 bg-white
          dark:bg-gray-900 px-4 pb-4">
          {navLinks.map(link => (
            <li key={link.href}>
              <a href={link.href}
                onClick={() => setMenuOpen(false)}
                className="block py-2 text-gray-600
                  dark:text-gray-300
                  hover:text-blue-600">
                {link.label}
              </a>
            </li>
          ))}
        </ul>
      )}
    </header>
  );
}
```

Uygulama Adımı

Uygulama-4: Hero Bölümü

src/components/sections/Hero.tsx:

```
export default function Hero() {
  return (
    <section id="hero"
      className="min-h-[80vh] flex items-center
        justify-center bg-gradient-to-br
        from-blue-50 to-indigo-100
        dark:from-gray-900 dark:to-gray-800
        px-4">
```

```
      <div className="text-center max-w-2xl">
        <p className="text-blue-600
          dark:text-blue-400 font-medium mb-2">
          Merhaba, ben
        </p>
        <h1 className="text-4xl md:text-6xl
          font-bold text-gray-900
          dark:text-white mb-4">
          Ad Soyad
        </h1>
        <p className="text-lg md:text-xl
          text-gray-600 dark:text-gray-300
          mb-8">
          Frontend Developer | React & TypeScript
        </p>
        <div className="flex gap-4
          justify-center flex-wrap">
          <a href="#projects"
            className="bg-blue-600 text-white
              px-6 py-3 rounded-lg font-medium
              hover:bg-blue-700 transition-colors">
            Projelerimi Gor
          </a>
          <a href="#contact"
            className="border border-blue-600
              text-blue-600 dark:text-blue-400
              px-6 py-3 rounded-lg font-medium
              hover:bg-blue-50
              dark:hover:bg-gray-800
              transition-colors">
            Iletisime Gec
          </a>
        </div>
      </div>
    </section>
  );
}
```

Uygulama Adımı

Uygulama-5: ProjectFilter Component'i

src/components/forms/ProjectFilter.tsx:

```
import type {
  Category, SortField, SortOrder
} from "../../types/project";

interface ProjectFilterProps {
  search: string;
  onSearchChange: (value: string) => void;
```

```
  category: Category | "all";
  onCategoryChange: (
    value: Category | "all"
  ) => void;
  sortField: SortField;
  onSortFieldChange: (value: SortField) => void;
  sortOrder: SortOrder;
  onSortOrderChange: (value: SortOrder) => void;
  resultCount: number;
  totalCount: number;
}

const categories: {
  value: Category | "all";
  label: string;
}[] = [
  { value: "all", label: "Tumu" },
  { value: "frontend", label: "Frontend" },
  { value: "fullstack", label: "Full Stack" },
  { value: "backend", label: "Backend" },
];

export default function ProjectFilter({
  search,
  onSearchChange,
  category,
  onCategoryChange,
  sortField,
  onSortFieldChange,
  sortOrder,
  onSortOrderChange,
  resultCount,
  totalCount,
}: ProjectFilterProps) {
  return (
    <div className="space-y-4 mb-8">
      {/* Arama */}
      <div className="relative">
        <input type="text"
          value={search}
          onChange={e =>
            onSearchChange(e.target.value)}
          placeholder="Proje ara (baslik,
            aciklama, teknoloji)..."
          className="w-full border border-gray-300
            dark:border-gray-600 rounded-lg
            px-4 py-2 pl-10
            dark:bg-gray-800 dark:text-white
            focus:ring-2 focus:ring-blue-500
            focus:border-transparent"
```

```
          aria-label="Proje ara" />
        <span className="absolute left-3
          top-1/2 -translate-y-1/2
          text-gray-400">
          &#128269;
        </span>
      </div>

      {/* Kategori + Siralama */}
      <div className="flex flex-col sm:flex-row
        gap-4 items-start sm:items-center
        justify-between">

        {/* Kategori butonlari */}
        <div className="flex gap-2 flex-wrap"
          role="group"
          aria-label="Kategori filtresi">
          {categories.map(cat => (
            <button key={cat.value}
              onClick={() =>
                onCategoryChange(cat.value)}
              className={`px-3 py-1.5 rounded-full
                text-sm font-medium
                transition-colors
                ${category === cat.value
                  ? "bg-blue-600 text-white"
                  : "bg-gray-100 dark:bg-gray-800
                     text-gray-600
                     dark:text-gray-300
                     hover:bg-gray-200
                     dark:hover:bg-gray-700"
                }`}
              aria-pressed={
                category === cat.value
              }>
              {cat.label}
            </button>
          ))}
        </div>

        {/* Siralama */}
        <div className="flex gap-2
          items-center">
          <select value={sortField}
            onChange={e => onSortFieldChange(
              e.target.value as SortField
            )}
            className="border border-gray-300
              dark:border-gray-600 rounded-lg
              px-3 py-1.5 text-sm
```

```
              dark:bg-gray-800 dark:text-white"
            aria-label="Siralama alani">
            <option value="year">Yil</option>
            <option value="title">Baslik</option>
          </select>
          <button
            onClick={() => onSortOrderChange(
              sortOrder === "asc" ? "desc" : "asc"
            )}
            className="border border-gray-300
              dark:border-gray-600 rounded-lg
              px-3 py-1.5 text-sm
              hover:bg-gray-100
              dark:hover:bg-gray-700
              dark:text-white"
            aria-label={`Siralama yonu:
              ${sortOrder === "asc"
                ? "artan" : "azalan"}`}>
            {sortOrder === "asc"
              ? "\u2191 Artan" : "\u2193 Azalan"}
          </button>
        </div>
      </div>

      {/* Sonuc sayisi */}
      <p className="text-sm text-gray-500
        dark:text-gray-400">
        {resultCount} / {totalCount} proje
        gosteriliyor
      </p>
    </div>
  );
}
```

Uygulama Adımı

Uygulama-6: ProjectList Section Component'i

src/components/sections/ProjectList.tsx:

```
import {
  useState, useEffect, useMemo
} from "react";
import type {
  Project, Category, SortField, SortOrder
} from "../../types/project";
import { fetchProjects } from
  "../../services/projectService";
import { applyFilters } from
  "../../utils/projectHelpers";
import ProjectFilter from
```

```
  "../forms/ProjectFilter";
import Card from "../ui/Card";

export default function ProjectList() {
  // State
  const [projects, setProjects] =
    useState<Project[]>([]);
  const [search, setSearch] = useState("");
  const [category, setCategory] =
    useState<Category | "all">("all");
  const [sortField, setSortField] =
    useState<SortField>("year");
  const [sortOrder, setSortOrder] =
    useState<SortOrder>("desc");
  const [loading, setLoading] = useState(true);
  const [error, setError] =
    useState<string | null>(null);

  // Veri cekme
  useEffect(() => {
    async function load() {
      try {
        setLoading(true);
        setError(null);
        const data = await fetchProjects();
        setProjects(data);
      } catch (err) {
        setError(
          err instanceof Error
            ? err.message
            : "Bilinmeyen hata olustu"
        );
      } finally {
        setLoading(false);
      }
    }
    load();
  }, []);

  // Filtrelenmis ve siralanmis veri (memoized)
  const filtered = useMemo(
    () => applyFilters(
      projects, search, category,
      sortField, sortOrder
    ),
    [projects, search, category,
     sortField, sortOrder]
  );

  return (
```

```
    <section id="projects" className="py-16 px-4">
      <div className="max-w-6xl mx-auto">
        <h2 className="text-3xl font-bold
          text-gray-900 dark:text-white mb-2">
          Projelerim
        </h2>
        <p className="text-gray-600
          dark:text-gray-400 mb-8">
          Uzerinde calistigim projeler
        </p>

        {/* Hata */}
        {error && (
          <div className="bg-red-50 border
            border-red-200 rounded-lg p-4 mb-6">
            <p className="text-red-800">{error}</p>
            <button
              onClick={() => window.location
                .reload()}
              className="text-sm text-red-600
                underline mt-2">
              Tekrar dene
            </button>
          </div>
        )}

        {/* Filtreler */}
        {!loading && !error && (
          <ProjectFilter
            search={search}
            onSearchChange={setSearch}
            category={category}
            onCategoryChange={setCategory}
            sortField={sortField}
            onSortFieldChange={setSortField}
            sortOrder={sortOrder}
            onSortOrderChange={setSortOrder}
            resultCount={filtered.length}
            totalCount={projects.length}
          />
        )}

        {/* Yukleniyor */}
        {loading && (
          <div className="flex justify-center
            py-12">
            <div className="animate-spin
              rounded-full h-8 w-8
              border-b-2 border-blue-600" />
          </div>
```

```
        )}

        {/* Bos durum */}
        {!loading && !error &&
          filtered.length === 0 && (
          <p className="text-center text-gray-500
            py-12">
            Eslesen proje bulunamadi.
          </p>
        )}

        {/* Proje grid */}
        <div className="grid grid-cols-1
          sm:grid-cols-2 lg:grid-cols-3 gap-6">
          {filtered.map(project => (
            <article key={project.id}
              className="bg-white dark:bg-gray-800
                rounded-lg border border-gray-200
                dark:border-gray-700 overflow-hidden
                hover:shadow-lg transition-shadow">

              {/* Gorsel */}
              <div className="h-48 bg-gradient-to-br
                from-blue-100 to-indigo-100
                dark:from-gray-700
                dark:to-gray-600 flex
                items-center justify-center">
                <span className="text-4xl
                  opacity-50">
                  &#128187;
                </span>
              </div>

              {/* Icerik */}
              <div className="p-4">
                <div className="flex items-start
                  justify-between mb-2">
                  <h3 className="font-bold
                    text-gray-900
                    dark:text-white">
                    {project.title}
                  </h3>
                  {project.featured && (
                    <span className="bg-yellow-100
                      text-yellow-800 text-xs
                      px-2 py-0.5 rounded-full">
                      One Cikan
                    </span>
                  )}
                </div>
```

```
                  <p className="text-sm text-gray-600
                    dark:text-gray-300 mb-3">
                    {project.description}
                  </p>
                  <div className="flex flex-wrap
                    gap-1 mb-3">
                    {project.tech.map(t => (
                      <span key={t}
                        className="bg-blue-100
                          dark:bg-blue-900
                          text-blue-800
                          dark:text-blue-200
                          text-xs px-2 py-0.5
                          rounded-full">
                        {t}
                      </span>
                    ))}
                  </div>
                  <p className="text-xs
                    text-gray-400">
                    {project.year}
                    {" \u00B7 "}
                    {project.category}
                  </p>
                </div>
              </article>
            ))}
          </div>
        </div>
      </section>
    );
}
```

Uygulama Adımı

Uygulama-7: App Component'i (Her Şeyi Birleştir)

`src/App.tsx`:

```
import Header from
  "./components/layout/Header";
import Footer from
  "./components/layout/Footer";
import Hero from
  "./components/sections/Hero";
import About from
  "./components/sections/About";
import Skills from
  "./components/sections/Skills";
import ProjectList from
```

```
  "./components/sections/ProjectList";
import ContactSection from
  "./components/sections/ContactSection";

export default function App() {
  return (
    <div className="min-h-screen bg-white
      dark:bg-gray-950 text-gray-900
      dark:text-white">
      <Header />

      <main>
        <Hero />
        <About />
        <Skills />
        <ProjectList />
        <ContactSection />
      </main>

      <Footer />
    </div>
  );
}
```

Not: `About`, `Skills`, `ContactSection` ve `Footer` component'lerini de aynı yapıda oluştur. `ContactSection` içinde `ContactForm` component'ini kullan. Her component kendi bölümünü yönetir; `App` yalnızca bir "orkestra şefi" gibi onları bir araya getirir.

Sırada Ne Var?

Checkpoint uygulamasının component yapısını tamamladık. Şimdi teslim sürecini ve **PR üzerinden kod inceleme** (code review) iş akışını öğrenelim.

Teorik Arka Plan

Pull Request nedir?
Pull Request (PR), bir branch'teki değişiklikleri ana branch'e (`main`) **birleştirme talebidir**. PR açıldığında:

- Tüm değişiklikler **satır satır** görüntülenir (diff).
- Ekip arkadaşları kodu **inceleyebilir**, yorum bırakabilir, değişiklik isteyebilir.
- Tüm inceleme tamamlanınca PR **merge** edilir (birleştirilir).

Kod incelemenin faydaları:

- **Hata tespiti:** İkinci bir göz, senin kaçırdığın hataları yakalar.

- **Bilgi paylaşımı:** Arkadaşın kodunu okuyarak yeni teknikler öğrenirsin.
- **Kod tutarlılığı:** Ekip genelinde ortak stil ve kalıplar oluşur.
- **Sorumluluk:** Merge edilen koddan **iki kişi** sorumludur — yazan ve onaylayan.

Uygulama Adımı

Uygulama-8: PR Oluşturma ve Kod İnceleme İş Akışı

1. Yeni branch oluştur ve commit at:

```
git checkout -b feature/checkpoint-1

# Adim adim commit
git add src/types/
git commit -m "feat: add project TypeScript types"

git add src/services/ src/utils/
git commit -m "feat: add data service and helpers"

git add src/components/layout/
git commit -m "feat: add Header and Footer layout"

git add src/components/sections/Hero.tsx
git add src/components/sections/About.tsx
git add src/components/sections/Skills.tsx
git commit -m "feat: add Hero, About, Skills
  sections"

git add src/components/forms/
git commit -m "feat: add ContactForm and
  ProjectFilter"

git add src/components/sections/ProjectList.tsx
git commit -m "feat: add ProjectList with
  filter/sort"

git add src/components/sections/ContactSection.tsx
git commit -m "feat: add ContactSection with
  form integration"

git add src/App.tsx
git commit -m "feat: integrate all sections
  in App"
```

2. GitHub'a push et:

```
git push -u origin feature/checkpoint-1
```

3. GitHub'da PR oluştur:

```
# GitHub CLI ile (veya github.com uzerinden)
gh pr create \
  --title "feat: Checkpoint #1 - Portfolio app" \
```

```
  --body "## Ozet
- Portfolyo landing sayfasi (Hero, About, Skills)
- Proje listesi (JSON fetch, filtre, siralama)
- Iletisim formu (controlled, validation)
- Responsive tasarim (mobil, tablet, masaustu)
- TypeScript (any kullanilmadi)

## Test plani
- [ ] Tum sayfalar mobilde duzgun gorunuyor
- [ ] Proje filtresi ıcalisyor
- [ ] Form dogrulamasi calisiyor
- [ ] TypeScript hatasi yok (npx tsc --noEmit)"
```

4. **Arkadaşını reviewer olarak ekle:**
   GitHub PR sayfasında sağ taraftaki "Reviewers" bölümünden sınıf arkadaşını seç. Arkadaşın kodu inceleyecek ve onaylayacak.
5. **Kod inceleme yaparken dikkat edilecekler:**
   - TypeScript tipleri doğru mu? `any` kullanılmış mı?
   - Component'ler tek sorumluluk ilkesine uyuyor mu?
   - Props interface'leri tanımlı mı?
   - key prop'ları doğru kullanılmış mı?
   - Form doğrulaması çalışıyor mu?
   - Responsive tasarım mobilde düzgün mü?
   - Erişilebilirlik (a11y): label, aria, semantik HTML var mı?
   - Kod okunabilir mi? Gereksiz tekrar var mı?
6. Arkadaşın "Approve" verdikten sonra PR'ı merge et:

```
# GitHub CLI ile
gh pr merge --squash

# veya GitHub web arayuzunden
# "Merge pull request" butonuna tikla
```

Derinlemesine Anlama

İyi bir code review nasıl yapılır?

- **Yapıcı ol:** "Bu kod kötü" yerine "Bu kısmı şu şekilde yazarsak daha okunabilir olur, çünkü..." de.
- **Soru sor:** "Bu neden böyle?" sorusu hem öğretir hem öğretir.
- **Olumlu geri bildirim ver:** "Bu yaklaşım çok temiz, beğendim!" motivasyon sağlar.
- **Küçük PR'lar tercih et:** 500 satırlık PR incelemesi verimli; 5000 satırlık PR "LGTM" onayı alır.
- **Otomatik kontrollere güven:** TypeScript derlemesi, linting, format kontrolü otomatik olmalıdır. İnsan gözü **mantık** ve **tasarım** hatalarına odaklanmalıdır.

Dikkat – Sık Yapılan Hatalar

- PR'da **kendi kodunu kendi onaylama** (self-approve). En az 1 arkadaş review'u **zorunludur**.
- Review beklerken başka bir branch'te çalışmaya devam edebilirsin: `git checkout -b feature/next-task`.
- Merge conflict çıkarsa **panik yapma**. Git, çakışan dosyaları işaretler; sen hangi değişikliğin kalacağına karar verirsin.
- PR açıklamasını **detaylı yaz**. "Değişiklikler" yazmak yeterli değildir; ne yaptığını, neden yaptığını ve nasıl test edileceğini belirt.

Sırada Ne Var?

PR tabanlı teslim sürecini öğrendik. Şimdi sınıf içi demo ve değerlendirme kriterlerini inceleyelim.

Teorik Arka Plan

Demo süreci (kişi başı ∼3–5 dakika):

1. **Proje tanıtımı (30 sn):** "Bu bir kişisel portföy uygulaması. React + TypeScript + Tailwind kullanıyorum."
2. **Responsive gösterim (1 dk):** Tarayıcıyı küçülterek mobil, tablet, masaüstü görünümlerini göster.
3. **Proje listesi ve filtre (1 dk):** Arama yap, kategori değiştir, sıralama uygula. "Veriyi JSON'dan fetch ediyorum, filtreleme useMemo ile optimize edilmiş."
4. **İletişim formu (1 dk):** Boş gönder (hata mesajları), geçersiz e-posta gir, doğru bilgilerle gönder (başarı mesajı).
5. **Kod yapısı (30 sn):** VS Code'da dosya yapısını göster. "Component'ler, tipler, servisler ve utility'ler ayrı klasörlerde."
6. **Soru-cevap (1 dk):** Eğitmenin veya arkadaşların soruları.

Gerçek Hayatta Karşılığı

Profesyonel yazılım dünyasında da benzer "demo" süreçleri vardır: Sprint review, stakeholder demo, design critique… Çalışan bir ürünü göstermek ve geri bildirim almak, yazılım geliştirmenin ayrılmaz parçasıdır. Bu beceriyi şimdiden kazanmak, iş hayatında büyük avantaj sağlar.

Kontrol Listesi

Teslim öncesi kontrol listesi:

- ☐ En az 5 React component oluşturulmuş mu? (Header, Hero, ProjectList, ProjectCard, ContactForm vb.)
- ☐ Props TypeScript interface'leri ile tiplenmiş mi?
- ☐ `useState` ile en az 3 farklı state yönetiliyor mu?
- ☐ `useMemo` en az 1 yerde kullanılmış mı? (filtreleme/sıralama)
- ☐ Controlled form çalışıyor mu? (value + onChange)
- ☐ Form doğrulaması (validation) uygulanmış mı?
- ☐ Liste render'da `.map()` ve doğru `key` prop kullanılmış mı?
- ☐ JSON'dan veri çekme (fetch) çalışıyor mu?
- ☐ Hata ve yükleniyor durumları gösterilmiş mi?
- ☐ Responsive tasarım çalışıyor mu? (mobil, tablet, masaüstü)
- ☐ `any` tipi **hiçbir yerde** kullanılmamış mı?
- ☐ TypeScript derlemede hata yok mu? (`npx tsc --noEmit`)
- ☐ PR açılmış ve en az 1 arkadaş review'u alınmış mı?
- ☐ En az 5 anlamlı commit atılmış ve branch kullanılmış mı?
- ☐ Sınıf içi demo hazır mı?

Teorik Arka Plan

S: Component ne zaman ayrılmalı?
C: Şu durumlarda yeni component oluştur:

- Bir bölüm birden fazla yerde kullanılıyorsa (yeniden kullanılabilirlik).
- Dosya 100–150 satırı aşıyorsa (okunabilirlik).
- Bir bölümün kendi state'i varsa (ayrıştırma).
- Test etmek istediğin bağımsız bir mantık varsa.

S: State nerede tutulmalı?
C: "State'i, onu kullanan en üst component'te tut" kuralını uygula (lifting state up). `search` state'ini hem `FilterBar` hem `ProjectGrid` kullanıyorsa, ikisinin ortak parent'ı olan `ProjectList`'te tut.
S: useEffect neden gerekli? Fetch'i doğrudan component'te çağıramaz mıyım?
C: Component her render'da baştan çalışır. Fetch'i doğrudan yazarsan **her render'da** istek gider — sonsuz döngü. `useEffect` yalnızca **belirtilen koşullarda** (bağımlılık dizisi) çalışır; boş dizi `[]` "yalnızca ilk render'da çalış" demektir.
S: key olarak index kullanmak neden sorun?
C: Listede ekleme, silme veya sıralama yapıldığında indeksler kayar. React, key=2'nin "aynı eleman" olduğunu sanır ama aslında farklı bir veridir. Bu, yanlış DOM güncellemelerine ve görsel hatalara yol açar. Her zaman benzersiz ve sabit bir `id` kullan.
S: useMemo her yerde kullanmalı mıyım?
C: Hayır. `useMemo`'nun kendisi de bir maliyet taşır (bağımlılık karşılaştırması, referans saklama). Yalnızca **ölçülebilir** performans sorunu olan hesaplamalarda kullan. Küçük dizilerde (<50 eleman) fark edilmez bile.

Kendini Test Et

Aşağıdaki sorulara kendi cümlelerinle 2–3 satır cevap yazabiliyor musun? Yazamıyorsan ilgili bölümü tekrar oku.

1. **React component nedir? Fonksiyonel component nasıl tanımlanır?**
2. **Props nedir? Props'un "tek yönlü veri akışı" (one-way data flow) özelliği ne anlama gelir?**
3. **Child component'ten parent'a nasıl "bilgi gönderilir"? Callback prop nedir?**
4. **useState hook'u ne döndürür? `setCount(count + 1)` ile `setCount(prev => prev + 1)` arasındaki fark nedir?**
5. **State güncellemesi neden immutable olmalıdır? `arr.push()` yerine ne kullanılır?**
6. **useMemo ne yapar? Ne zaman kullanılır, ne zaman gereksizdir?**
7. **Controlled form ile uncontrolled form arasındaki fark nedir? React'te hangisi tercih edilir ve neden?**
8. **`.map()` ile liste render ederken `key` prop'u neden zorunludur? key olarak index kullanmak neden sorunludur?**

9. **Pull Request (PR) nedir? Kod incelemenin (code review) faydaları nelerdir?**
10. **"Lifting state up" kuralı ne demektir? State hangi component'te tutulmalıdır?**

- https://react.dev/learn
- https://react.dev/learn/describing-the-ui
- https://react.dev/learn/adding-interactivity
- https://react.dev/learn/managing-state
- https://react-typescript-cheatsheet.netlify.app
- https://kentcdodds.com/blog/application-state-management-with-react
- https://docs.github.com/en/pull-requests
- https://google.github.io/eng-practices/review/

Sırada Ne Var?

Bu hafta React'in temel yapı taşlarını öğrendin: Component, props, state, controlled form, liste render. Checkpoint #1 ile tüm bu bilgileri çalışan bir uygulamada birleştirdin ve PR tabanlı kod inceleme iş akışını deneyimledin.

Gelecek hafta (LAB-7) React'in ileri konularına geçeceğiz:

- **useEffect** ile side effect yönetimi (API çağrıları, event listener'lar, cleanup).
- **Custom hooks** yazarak tekrarlayan mantığı soyutlama.
- **React Router** ile çok sayfalı uygulama (SPA routing).
- **Context API** ile global state yönetimi (tema, dil, kullanıcı bilgisi).

Temeller sağlam — artık üstüne inşa etme zamanı!